1 Approximation Algorithms

Approximation algorithms don't always return an optimal solution, but their solutions are often close to optimal. They always run in polynomial time, and they are often relatively simple.

1.1 Vertex Cover

Algorithm: Set $S = \emptyset$. If G has an edge $e = \{u, v\}$, add both endpoints of e to S, and remove u and v from G. (Note that when we remove a vertex from a graph, we also remove all edges incident to the vertex.) Repeat this process until G has no edges and return S.

Correctness: Notice that the edges considered by the algorithm form a matching M, so $|ALG| = 2 \cdot |M|$. Furthermore, each vertex in OPT can cover at most one edge in M, so $|OPT| \geq |M|$. Thus, $|ALG| = 2 \cdot |M| \leq 2 \cdot |OPT|$.

1.2 Load Balancing

Algorithm: The algorithm is greedy: iterate through the jobs (in any order), assigning each one to the machine with the smallest load so far.

Correctness: Suppose ALG achieves a makespan of T, the load of $i \in [m]$ is T, and k was the last job assigned to i. When ALG assigned k to i, i had the smallest load. This means the load on every machine is at least $T - \ell[k]$, so $\sum_{j=1}^{n} \ell[j] \geq m(T - \ell[k])$. Rearranging yields $T \leq \frac{\sum_{j=1}^{n} \ell[j]}{m} + \ell[k]$.

Now let T^* denote the optimal makespan. Since the average load in any assignment is $\frac{\sum_{j=1}^{n} \ell[j]}{m}$, we have $T^* \geq \frac{\sum_{j=1}^{n} \ell[j]}{m}$. Combining this with the inequality above, we get $T \leq \frac{\sum_{j=1}^{n} \ell[j]}{m} + \ell[k] \leq T^* + T^* = 2T^*$.